데이터베이스 프로그래밍 프로젝트

데이터베이스를 활용하여 생활 속 문제를 자동화하거나 개선하는 시스템을 직접 설계·구현하는 프로젝트입니다.

1. 프로젝트 개요

(데이터베이스를 활용하여 수동으로 처리되던 생활 속 문제를 자동화하거나 개선하는 시스템을 만듦 - 예: 도서 대여 시스템, 사물함 관리, 출결 관리, 대여장비 관리 등)

Military Strategic Weapons & Control System Database (MSWCS)

2. 프로젝트 주제 선정

(주제 선정 기준 : 수작업 환경에서 오류가 발생하는가, 자동화하면 효율이 좋아지는가, 데이터 흐름을 명확히 설계할 수 있는가?)

<작성 항목>

- 해결하려는 문제(현재 수작업 환경에서 발생하는 구체적인 문제점과 비효율성을 서술)

: 한 국가에서 군사 무기, 방어 시스템을 구축 하려 한다.
이때 데이터베이스 시스템을 사용하지 않으면 많은 문제점이 발생한다.
예를 들어, 한 군사 데이터에 변경이 있을때 모든 데이터의 수치가 일관되지 않는 것,
각 데이터별 접근 권한을 체계적으로 관리할 수 없는 문제 등등..
= 일관성 침해, 보안취약점 ..

- 자동화 필요성 (위 문제점을 해결하기 위해 데이터베이스 시스템이 왜 필요한지, 자동화 시 어떤 이점이 있는지 설명)

: 위에서 기술한 문제점은 데이터베이스 시스템을 사용하면 해결된다.
데이터베이스는 모든 데이터가 일관하므로 일관성이 보장되고,
각 데이터별 접근 권한을 다르게 설정해 기존 방식에서 생기는 보안 취약점이 사라진다.
군사데이터는 한 일을 수행할때 많은 작업을 해야하므로 자동화한다면 기존보다 편리하고 신속해진다.
그 외에도 데이터베이스를 사용함으로써 생기는 모든 이점이 이유에 해당된다.

- 주제 선택 이유(이 프로젝트 주제를 선택한 배경 및 동기)

: 나는 보안과 백엔드 데이터베이스 서버에 관심이 많기 때문에 이를 융합하여
보안이 철저해야 하는 군사시스템을 선택했다.

3. 요구사항 정의

1. 인증 및 권한 관리

1.1 (사용자 식별**):** 시스템은 모든 접근 시도에 대해 사용자를 식별해야 한다.

1.2 (역할 기반 접근 제어**):** 시스템은 최소 3가지 역할(예: 관리자(Admin), 지휘관(Commander), 조작원(Operator))을 구분해야 한다.

1.3 (로그 권한 제어**):** operation_log는 어떤 유저도 삭제 및 수정이 불가해야 한다.

1.4 (권한 분리**):**

관리자(Admin):모든 역할의 권한을 상속하며, operation_log(SELECT, INSERT)를 제외한 모든 테이블의 등록 및 상태 변경(유지보수 등) 권한을 가진다.

감사관(Auditor): 모든 테이블을 조회할 권한을 가진다.

지휘관(Commander): Operator의 권한을 상속하며, control_code를 발급하고, operation_log와 weapon_system를 조회할 권한을 가진다.

조작원(Operator): control_code를 사용하여 작전을 수행하고, control_code와 weapon_system를 조회할 권한을 가진다.

2. 무기 시스템 관리 (Admin) **weapon_system** 테이블 관련 기능.

2.1 (시스템 등록**):** 관리자는 새로운 무기/방어 시스템(weapon_system)을 DB에 등록할 수 있어야 한다. (이름, 타입, 보안 등급 포함)

이때, weapon_system(id)는 중복되어선 안되고, 모든 열은 NOT NULL이어야 한다.

2.2 (상태 관리**):** 관리자는 기존 무기 시스템의 status를 'active', 'maintenance', 'standby'로 변경할 수 있어야 한다.

3. 제어 코드 관리 (Commander) **control_code** 테이블 관련 기능.

3.1 (코드 발급**):** 지휘관은 특정 weapon_system(ID)에 연결되는 control_code를 생성할 수 있어야 한다.

이때, control_code(id), control_code(code_hash)는 중복되어선 안되고, 모든 열은 NOT NULL이어야 한다.

3.2 (코드 해싱**):** 시스템은 control_code 원문이 아닌, 해시된 값(code_hash)만을 저장해야 한다.

3.3 (유효 기간 설정**):** 코드 발급 시, 반드시 valid_to (유효 종료) 시간을 설정해야 한다.

3.4 (유효기간 즉효력**):** 코드 발급 시, 반드시 valid_from (유효 시작)은 현재이어야 한다.

3.5 (기본 유효기간**):** 유효기간을 따로 지정하지 않는다면, 1년 뒤까지 자동 설정된다.

4. 작전 수행 및 로깅 (Operator) **operation_log** 테이블 관련 기능

4.1 (작전 수행 요청**):** 조작원은 'test' 또는 'launch' 등의 action을 특정 weapon_system에 대해 요청할 수 있어야 한다.

4.2 (제어 코드 검증**):** 시스템은 작전 수행 요청 시, 조작원이 제출한 control_code가 다음 3가지 조건을 모두 만족하는지 실시간으로 검증해야 한다.

control_code 테이블에 해시값이 존재하는가**?**

해당 코드가 요청된 **weapon_system_id**와 일치하는가**?**

현재 시간이 코드의 **valid_from**과 **valid_to** 사이인가**?**

위 조건을 충족하지 못한다면 수행 불가 응답을 전송하고 해당 시도를 기록해야 한다.

이때, operation_log(id)는 중복되어선 안되고, 모든 열은 NOT NULL이어야 한다.

4.3 (자동 로그 생성**):** 코드 검증이 성공하면, 시스템은 즉시 operation_log에 관련 기록(누가, 언제, 어떤 무기를, 어떤 코드로, 무슨 작업을 했는지)을 자동으로 생성해야 한다.

이때, operation_log(id)는 중복되어선 안되고, 모든 열은 NOT NULL이어야 한다.

4.4 (수행 거부**):** 코드 검증이 실패하면, 시스템은 작전 수행을 거부하고 해당 시도를 기록해야 한다.

이때, operation_log(id)는 중복되어선 안되고, 모든 열은 NOT NULL이어야 한다.

5. 감사 및 추적 (Commander / Auditor) **operation_log** 조회 관련 기능

5.1 (로그 조회**):** 지휘관(또는 감사관)은 operation_log의 모든 기록을 조회할 수 있어야 한다.

5.2 (로그 필터링): 시스템은 감사관이 특정 조건(예: 특정 weapon_system_id별, 특정 operator별, 특정 action별, 기간별)으로 로그를 검색하고 필터링할 수 있는 기능을 제공해야 한다.

4. 데이터베이스 설계

4-1. 테이블 구조 설계

테이블명	주요 속성(컬럼명)	역할 및 주요 제약조건
personnel	id PK INT 고유 ID uuid UK INT 외부용 고유 ID name VARCHAR(100) 이름 rank ENUM('Commander', 'Operator', 'Admin', 'Auditor') 계급, 역할 clearance_level INT 보안 등급 (1~3) status ENUM('active', 'on_leave', 'retired') DEFAULT 'active' 인원의 복무 상태	인원 접근 권한 등의 인원 관리
weapon_system	id PK INT 고유 ID uuid UK INT 외부용 고유 ID name VARCHAR(50) 무기/방어 시스템 명 type ENUM('missile','drone','radar','defense_grid') 종류 status ENUM('active','maintenance','standby') 상태 security_level INT 보안 등급 (1~5, 1=최고)	무기/방어 시스템 정보 저장
control_code	id PK INT 고유 ID uuid UK INT 외부용 고유 ID weapon_system_id FK → weapon_system(id) INT 연결 무기 code_hash CHAR(64) SHA-256 해시 형태 인증 코드 issued_by_id FK → personnel(id) INT 발급자 ID valid_from DATETIME 사용 가능 시작 시간	발사/운영 인증 코드 저장

	valid_to DATETIME 사용 가능 종료 시간	
operation_log	id PK INT 고유 ID uuid UK INT 외부용 고유 ID weapon_system_id FK → weapon_system(id) INT 연결 무기 control_code_id FK → control_code(id) INT 사용된 코드 operator_id FK → personnel(id) INT 조작자 ID timestamp DATETIME 기록 시각 action ENUM('test','launch','maintenance') 수행 작업 result ENUM('succeeded','failed') 결과	무기 시스템 운영/발사/테스트 기록
control_code_archive	id PK INT 고유 ID code_id PK INT 만료된 코드의 원본 아이디 uuid UK INT 만료된 코드의 외부용 고유 ID weapon_system_id INT 만료된 코드의 weapon_system(id) INT 연결 무기 code_hash CHAR(64) 만료된 코드의 SHA-256 해시 형태 인증 코드 issued_by_id INT 만료된 코드의 발급자 valid_from DATETIME 만료된 코드의 유효 시작 날짜 valid_to DATETIME 만료된 코드의 유효 만료 날짜 archived_at DATETIME 만료된 코드가 아카이브 된 시점	

4-2. 테이블 간 관계(ERD)

personnel과 control_code 1(인원):N(발급 코드)

weapon_system과 control_code 1(무기):1(제어 코드)

personnel과 operation_log 1(인원):N(사용 로그)

weapon_system과 operation_log 1(무기):N(사용 로그)

control_code와 operation_log 1(제어 코드):N(사용 로그)

5. 기능 구현

- 프로시저
- 만료된 코드 자동으로 아카이브화 및 기존 테이블에서 제거
- 작전 수행 및 수행 시 3가지 조건 검증

- Commander(지휘관) control_code 생성
- 감사관 필터링
- 트리거
- 컨트롤 코드가 INSERT 될 때, 해쉬 검사 및 유효날짜 무결성 검사
- personnel 테이블에 INSERT 될 때, db에 등록된 유저인지 확인

- 프로시저***(Procedure)***로 여러 ***SQL*** 문장을 하나의 기능으로 묶기
*(*예시 **-** 학생이 회의실 예약을 요청할 때, 예약 등록 전, 동일한 *RoomID, ReqDate,* 예약 시간대에 기존 예약이 있는지 *ROOM_RESERVATION* 테이블을 조회하여 이중 예약 여부를 검사, 검사 결과 충돌이 없다면, *ROOM_RESERVATION* 테이블에 새로운 예약 기록을 *INSERT*함*)*
- 트리거***(Trigger)***로 데이터 무결성 자동 검사
*(*예시 **-** 예약 시작시간이 종료시간보다 같거나 늦을 경우 해당 작업을 자동으로 취소하고 오류 메시지를 발생시킴*)*
- 뷰***(View)***로 조회 기능 단순화
*(*예시 **-** *ROOM_RESERVATION* 테이블과 *ROOM* 테이블, 그리고 *STUDENT* 테이블을 *JOIN*하여, '회의실명', '예약일자', '시작시간', '예약자 이름', '예약자 학과'등 필요한 정보만을 조합한 가상 테이블을 생성*)*

6. 테스트 및 검증

보안 및 기능 종합 테스트 쿼리

0. 환경 설정 및 데이터 초기화

```
-- 환경 설정 쿼리가 실행되었다고 가정하고 데이터만 초기화합니다.

USE MSWCS;


-- 기존 데이터 초기화 (테스트의 일관성을 위해)

DELETE FROM operation_log;

DELETE FROM control_code_archive;


-- control_code 테이블 초기화 (FK 제약 때문에 operation_log 다음 순서)

DELETE FROM control_code;


-- personnel 및 weapon_system 초기화 (FK 제약 때문에 가장 먼저 비워야 함)

DELETE FROM personnel;

DELETE FROM weapon_system;
```

```
-- AUTO_INCREMENT 값 초기화
ALTER TABLE personnel AUTO_INCREMENT = 1;
ALTER TABLE weapon_system AUTO_INCREMENT = 1;
ALTER TABLE control_code AUTO_INCREMENT = 1;
ALTER TABLE operation_log AUTO_INCREMENT = 1;

-- 기본 데이터 재삽입
INSERT INTO personnel (name, `rank`, clearance_level, status)
VALUES
('operator_kim', 'Operator', 3, 'active'), -- ID 1
('commander_lee', 'Commander', 2, 'active'), -- ID 2
('admin_choi', 'Admin', 1, 'active'), -- ID 3
('auditor_park', 'Auditor', 2, 'active'); -- ID 4

INSERT INTO weapon_system (name, type, status, security_level)
VALUES
('Aegis-01', 'radar', 'active', 3), -- ID 1
('Drone-X7', 'drone', 'standby', 2), -- ID 2
('Missile-K2', 'missile', 'active', 1); -- ID 3

-- 유효한 제어 코드 삽입 (ID 1, 2, 3)
INSERT INTO control_code (
    weapon_system_id, code_hash, issued_by_id, valid_from, valid_to
)
VALUES
(1, SHA2('CODE_AEGIS_VALID', 256), 2,
    DATE_SUB(NOW(), INTERVAL 1 DAY), DATE_ADD(NOW(), INTERVAL 1 DAY)), -- ID 1 (현재 유효)
(2, SHA2('CODE_DRONE_VALID', 256), 2,
```

```
    DATE_SUB(NOW(), INTERVAL 1 DAY), DATE_ADD(NOW(), INTERVAL 1 MONTH)), -- ID 2 (현재 유효)
(3, SHA2('CODE_MISSILE_EXPIRED', 256), 2,
    DATE_SUB(NOW(), INTERVAL 3 DAY), DATE_SUB(NOW(), INTERVAL 1 DAY)); -- ID 3 (만료됨)

SELECT '--- 0. 초기 데이터 세팅 완료 ---' AS Test_Status;
```

---

1. 데이터 무결성 및 트리거 검증

```
SELECT '--- 1.1 trg_code_insert (유효기간) 검증 ---' AS Test_Status;

-- 1-1-A. [오류 예상] valid_to가 valid_from보다 빠를 때
-- SIGNAL 오류 예상: "valid_to는 valid_from 보다 이를 수 없습니다."
INSERT INTO control_code (weapon_system_id, code_hash, issued_by_id, valid_from, valid_to)
VALUES (1, SHA2('INVALID_DATE_TEST', 256), 2, NOW(), DATE_SUB(NOW(), INTERVAL 1 DAY));

-- 1-1-B. [오류 예상] 해시 포맷이 64자 헥스가 아닐 때
-- SIGNAL 오류 예상: "해시 포맷이 맞지 않습니다: 64개의 헥스 문자를 추가하세요 (SHA-256)"
INSERT INTO control_code (weapon_system_id, code_hash, issued_by_id, valid_from, valid_to)
VALUES (1, 'short_hash', 2, NOW(), DATE_ADD(NOW(), INTERVAL 1 DAY));


SELECT '--- 1.2 trg_check_db_user_name_before_insert 검증 ---' AS Test_Status;

-- 1-2-A. [오류 예상] DB에 존재하지 않는 유저명을 삽입 시
-- SIGNAL 오류 예상: "personnel 테이블에 삽입된 이름이 현재 DB에 등록된 유저명이 아닙니다."
INSERT INTO personnel (name, `rank`, clearance_level)
VALUES ('non_existent_user', 'Operator', 3);
```

```
-- 1-2-B. [성공 예상] DB에 존재하는 유저명을 삽입 시 (테스트를 위해 임시로 유저 생성했다고 가정)

-- (실제 DB에 'test_user'@'localhost'가 없으면 1-2-A와 동일한 오류 발생)

-- 이 테스트는 root 권한으로 실행해야 유의미함. 현재는 생략.
```

---

2. 기능 프로시저 검증

2-1. archive_expired_control_codes 검증

```
SELECT '--- 2.1 archive_expired_control_codes (아카이빙/롤백) 검증 ---' AS Test_Status;

-- 테스트를 위해 세션 유저를 'admin_choi'@'localhost'로 설정해야 합니다.

-- SQL 환경에 따라 SET SESSION USER='admin_choi'@'localhost'; 또는 별도 접속 필요.


-- 2-1-A. [성공 예상] ID 3번 만료 코드 아카이브 및 삭제

-- ID 3번 코드는 '만료됨' (valid_to < NOW()) 상태

CALL archive_expired_control_codes();

SELECT '✅ 아카이브 후 control_code 수:' AS Status, COUNT(*) FROM control_code;          -- 예상: 2

SELECT '✅ 아카이브 후 control_code_archive 수:' AS Status, COUNT(*) FROM control_code_archive; -- 예상: 1


-- 2-1-B. [오류 유발 및 롤백 검증] (중복 키 오류 1062 유발)

-- 이미 아카이브된 ID 3번을 archive 테이블에 수동으로 다시 삽입

INSERT INTO control_code_archive (

    id, uuid, weapon_system_id, code_hash, issued_by_id, valid_from, valid_to

) VALUES (

    3, UUID(), 3, SHA2('DUMMY_HASH', 256), 2, NOW(), NOW()

);

-- ID 3번 코드를 다시 만료 상태로 삽입 (아카이빙을 시도할 코드)
```

```
INSERT INTO control_code (weapon_system_id, code_hash, issued_by_id, valid_from, valid_to)

VALUES (3, SHA2('CODE_MISSILE_EXPIRED_2', 256), 2, DATE_SUB(NOW(), INTERVAL 3 DAY),
DATE_SUB(NOW(), INTERVAL 1 DAY)); -- ID 4


-- CALL 실행 (중복 키 오류 발생 유도)

-- SIGNAL 오류 예상: "아카이빙 중 데이터 오류가 발생하여 트랜잭션이 롤백되었습니다."

CALL archive_expired_control_codes();


-- 2-1-C. [롤백 검증] ID 4번 코드가 삭제되지 않고 남아있는지 확인

SELECT '✅ 롤백 후 control_code에 ID 4가 남아있어야 함:' AS Status, COUNT(*) FROM control_code
WHERE id = 4; -- 예상: 1

SELECT '✅ 롤백 후 control_code_archive 수는 변동 없어야 함:' AS Status, COUNT(*) FROM
control_code_archive; -- 예상: 2 (원래 1 + 수동 삽입 1)


DELETE FROM control_code WHERE id = 4; -- 다음 테스트를 위해 정리
```

2-2. operation 검증

```
SELECT '--- 2.2 operation (작전 수행) 검증 ---' AS Test_Status;


-- 테스트를 위해 세션 유저를 'operator_kim'@'localhost'로 설정해야 합니다.

-- SQL 환경에 따라 SET SESSION USER='operator_kim'@'localhost'; 또는 별도 접속 필요.

-- 아래는 쿼리 실행만을 가정합니다. (operator_kim의 ID는 1, Aegis-01 ID는 1)

SET @AEGIS_HASH = SHA2('CODE_AEGIS_VALID', 256);

SET @DRONE_HASH = SHA2('CODE_DRONE_VALID', 256);


-- 2-2-A. [성공 예상] 유효한 코드와 ID로 작전 수행

-- 'operator_kim' (ID 1)이 Aegis-01 (ID 1)에 대해 ID 1 코드 사용

-- operation_log에 'succeeded' 기록 예상
```

```
CALL operation(1, @AEGIS_HASH, 'launch');

-- 아래 쿼리를 작동시키기위한 세션 유저가 role_commander, role_auditor 이상의 권한이어야
합니다.

SELECT result, action FROM operation_log ORDER BY id DESC LIMIT 1; -- 예상: succeeded, launch


-- 2-2-B. [오류 예상] weapon_system_id 불일치

-- SIGNAL 오류 예상: "weapon_system_id가 맞지 않습니다."

-- Aegis-01 코드를 Drone-X7 (ID 2)에 사용 시도

-- operation_log에 'failed' 기록 후 SIGNAL 발생 예상

CALL operation(2, @AEGIS_HASH, 'test');

-- 아래 쿼리를 작동시키기위한 세션 유저가 role_commander, role_auditor 이상의 권한이어야
합니다.

SELECT result, action FROM operation_log ORDER BY id DESC LIMIT 1; -- 예상: failed, test


-- 2-2-C. [오류 예상] 코드 존재하지 않음

-- SIGNAL 오류 예상: "코드가 control_code 테이블에 존재하지 않습니다."

CALL operation(1, SHA2('NON_EXISTENT_HASH', 256), 'maintenance');

SELECT '작전 로그에 실패 기록 없음 (SIGNAL이 먼저 발생)' AS Note;
```

2-3. create_control_code 검증

```
SELECT '--- 2.3 create_control_code (코드 생성) 검증 ---' AS Test_Status;


-- 테스트를 위해 세션 유저를 'commander_lee'@'localhost'로 설정해야 합니다.

-- commander_lee ID는 2


-- 2-3-A. [오류 예상] 유효기간이 남은 코드 중복 발급 시도

-- Drone-X7 (ID 2)에 이미 유효한 코드(ID 2)가 있음.

-- SIGNAL 오류 예상: "이 무기 시스템에 유효기간이 남은 활성 컨트롤 코드가 이미 존재합니다."
```

```
CALL create_control_code(2, SHA2('NEW_CODE_FOR_DRONE', 256), DATE_ADD(NOW(), INTERVAL 1 YEAR));


-- 2-3-B. [성공 예상] 유효한 새 코드 발급

-- Missile-K2 (ID 3)는 현재 유효한 코드가 없음.

CALL create_control_code(3, SHA2('NEW_CODE_FOR_MISSILE', 256), DATE_ADD(NOW(), INTERVAL 1
YEAR));

SELECT '✅ 새 코드 생성 확인:', code_hash FROM control_code ORDER BY id DESC LIMIT 1;
```

2-4. operation_log_filter 검증

```
SELECT '--- 2.4 operation_log_filter (감사관 필터) 검증 ---' AS Test_Status;

-- 테스트를 위해 세션 유저를 'auditor_park'@'localhost'로 설정해야 합니다.

-- SQL 환경에 따라 SET SESSION USER='auditor_park'@'localhost'; 또는 별도 접속 필요.


-- 2-4-A. [오류 예상] 허용되지 않은 컬럼으로 필터 시도

-- SIGNAL 오류 예상: "허용되지 않은 필터 컬럼입니다."

CALL operation_log_filter('invalid_column', 'some_value', 'timestamp', 'desc');


-- 2-4-B. [오류 예상] SQL Injection 시도 (Prepared Statement 방어)

-- SQL Injection 시도: ''' OR 1=1 --'

-- Prepared Statement 덕분에 SIGNAL은 발생 안함. 결과는 0행 예상 (방어 성공).

CALL operation_log_filter('action', ''' OR 1=1 --', 'timestamp', 'desc');

SELECT '✅ SQL Injection 방어 성공. 결과는 0행이어야 함.' AS Status;
```

7. 오류 해결 과정

취약점을 막으며 코딩하는것이 힘들었지만,

결과적으로 권한을 계층, 상속구조를 이용해 철저히하고,

operation_log에 로그 불변성을 적용해 SELECT, INSERT를 제외한 모든 조작을 불가능하게 했습니다.

모든 활동을 general_log 테이블에 기록해 보안을 강화했습니다.

인증 코드를 SHA256 해시를 이용해 복호화를 불가능하게 했습니다.

AUTO_INCREMENT를 이용한 공격을 막아내기위해

외부용 UUID를 모든 테이블에 설정했습니다.

DB내부에서 sql 인젝션등을 성공적으로 막아냈습니다.

또한 프로시저를 이용해 권한을 유저에게 주지 않고 기능을 사용하도록 했습니다.

8. 프로젝트 수행 후 배운 점

트리거와 프로시저의 사용법과 DB를 설계하는 법을 배웠으며

DB 보안을 설계하는 법 또한 배웠습니다

9. 전체 느낀 점

더 많은 기능을 추가할 가능성이 보이고,

DB 보안 설계에 대해 많은 것을 배웠습니다.